

AI-Assisted Data Visualization and Reasoning using LLMs (LIDA)

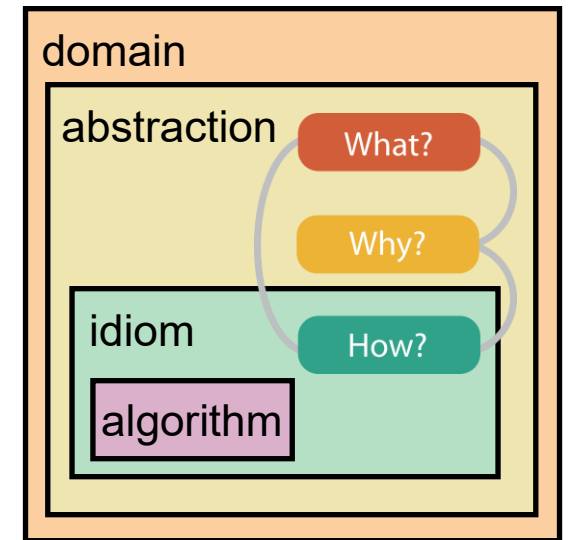


BDA LAB
IIT Allahabad

Visualization Analytics Framework

Analysis framework: Four levels, three questions

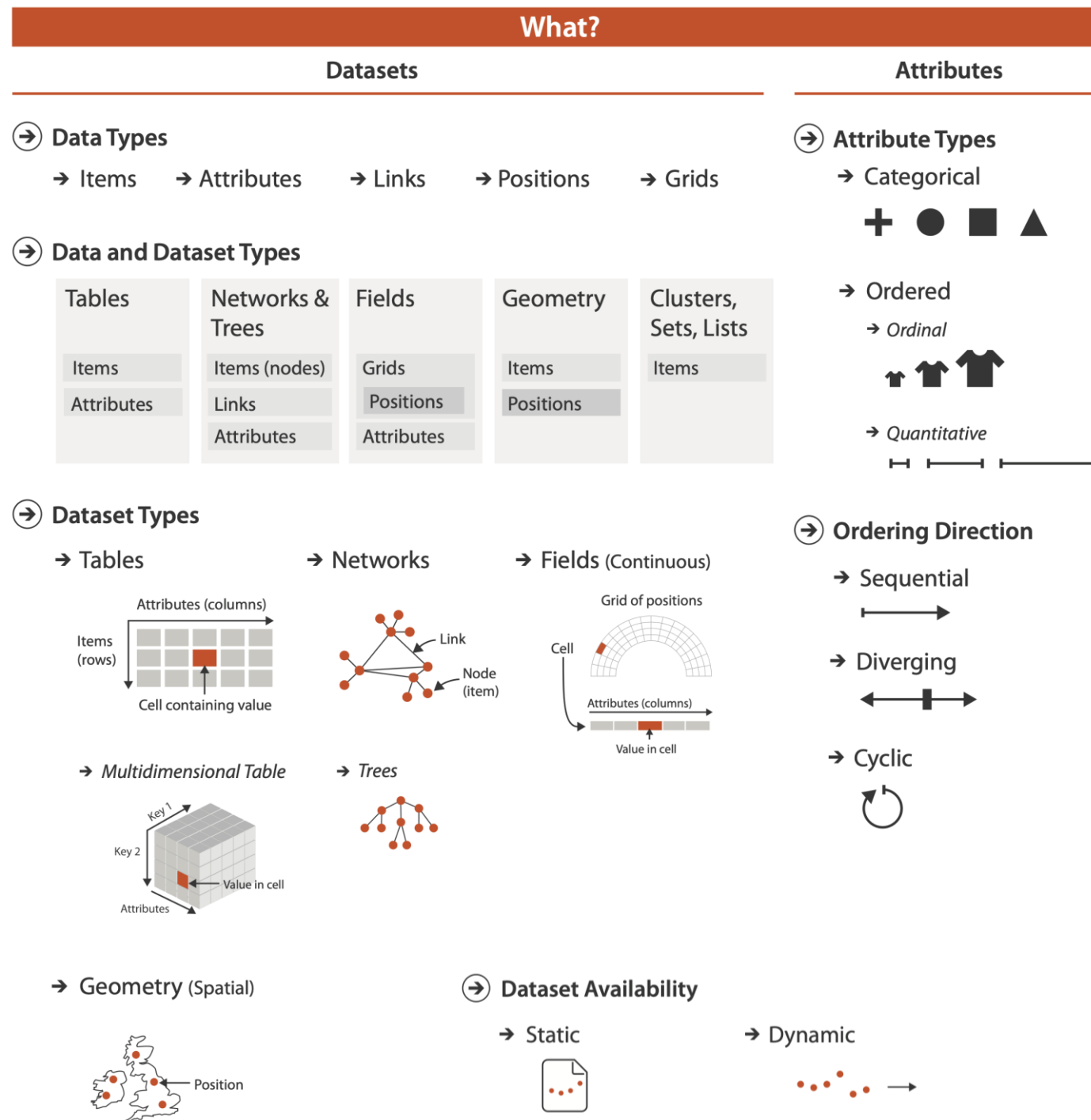
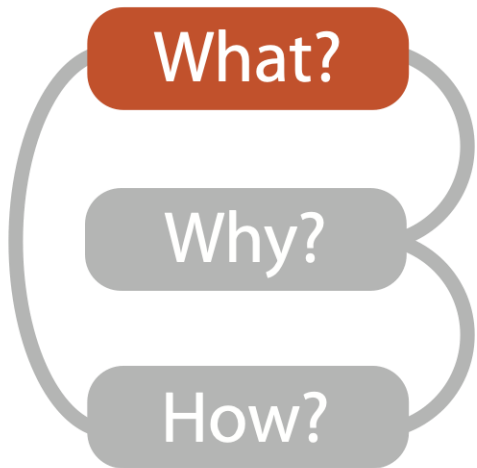
- **domain situation**
 - who are the target users?
- **abstraction**
 - translate from specifics of domain to vocabulary of vis
 - **what** is shown? **data** abstraction
 - **why** is the user looking at it? **task** abstraction
- **idiom**
 - **how** is it shown?
 - **visual encoding** idiom: how to draw
 - **interaction** idiom: how to manipulate
- **algorithm**
 - efficient computation



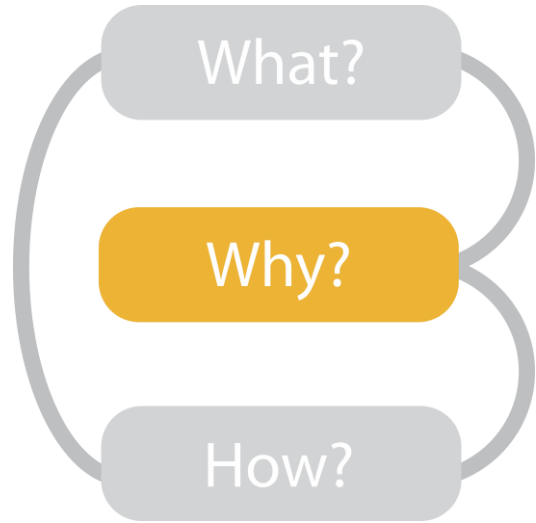
[A Multi-Level Typology of Abstract Visualization Tasks. Brehmer and Munzner. IEEE TVCG 19(12):2376-2385, 2013 (Proc. InfoVis 2013).]

[A Nested Model of Visualization Design and Validation. Munzner. IEEE TVCG 15(6):921-928, 2009 (Proc. InfoVis 2009).]

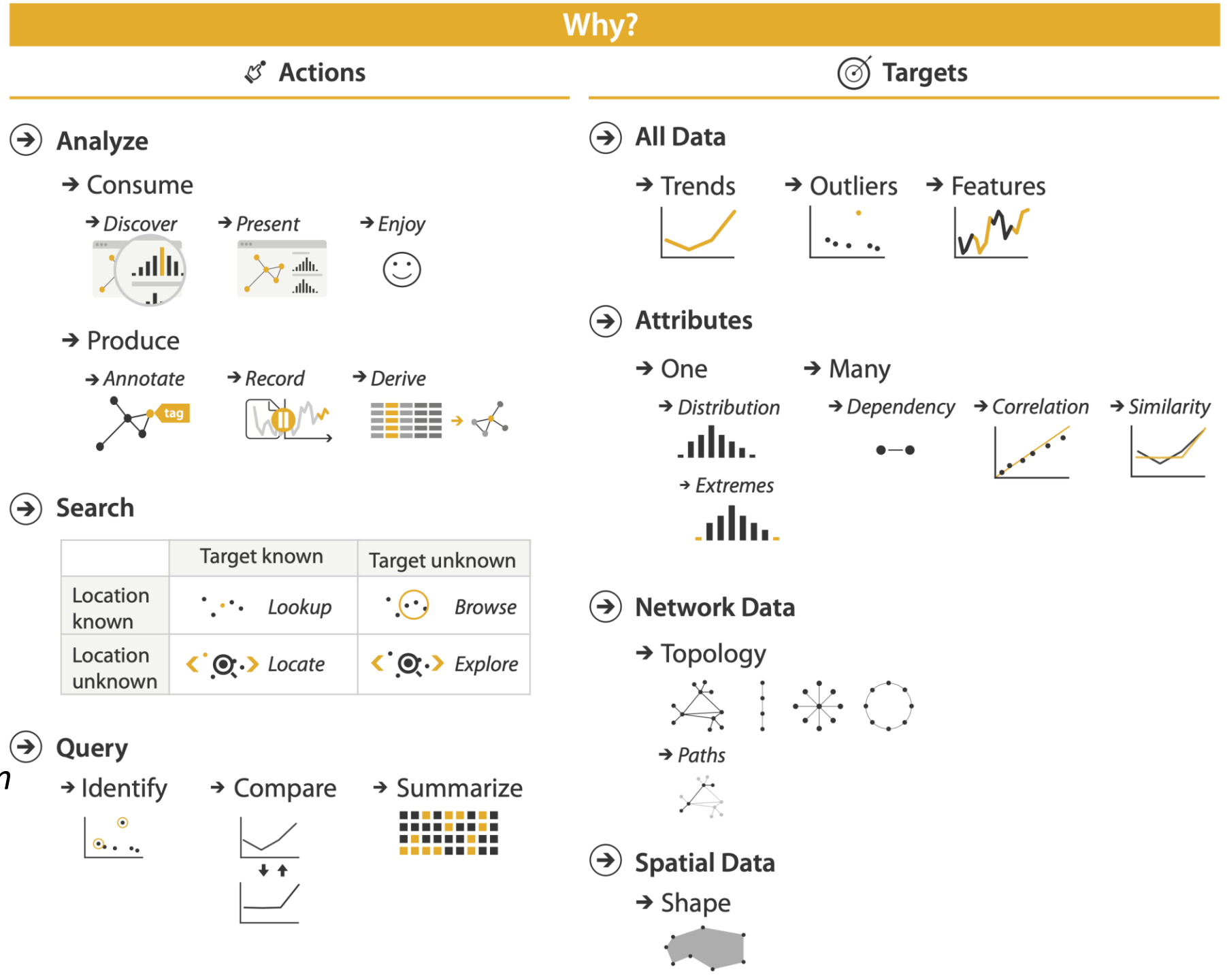
Analysis framework



Analysis framework



- {action, target} pairs
 - discover distribution
 - compare trends
 - locate outliers
 - browse topology



How?

Encode

➔ Arrange

➔ Express



➔ Separate



➔ Order



➔ Align



➔ Use



➔ Map

from **categorical** and **ordered** attributes

➔ Color

➔ Hue



➔ Saturation



➔ Luminance



➔ Size, Angle, Curvature, ...



➔ Shape



➔ Motion

Direction, Rate, Frequency, ...



Manipulate

➔ Change



➔ Select



➔ Navigate



Facet

➔ Juxtapose



➔ Partition



➔ Superimpose



Reduce

➔ Filter



➔ Aggregate



➔ Embed



What?

Why?

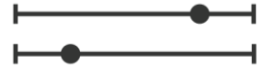
How?

Other Important Visualization Consideration

Channels: Rankings

➔ Magnitude Channels: Ordered Attributes

Position on common scale



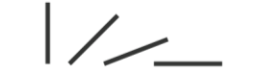
Position on unaligned scale



Length (1D size)



Tilt/angle



Area (2D size)



Depth (3D position)



Color luminance



Color saturation



Curvature



Volume (3D size)



Best

Effectiveness

Least

➔ Identity Channels: Categorical Attributes

Spatial region



Color hue



Motion



Shape



- expressiveness
 - match channel and data characteristics
- effectiveness
 - channels differ in accuracy of perception

Grouping

- **containment**
- **connection**

- **proximity**
 - same spatial region
- **similarity**
 - same values as other

Marks as Links

categorical channels

➔ **Containment**



➔ **Connection**



➔ **Identity Channels: Categorical Attributes**

Spatial region



Color hue



Motion



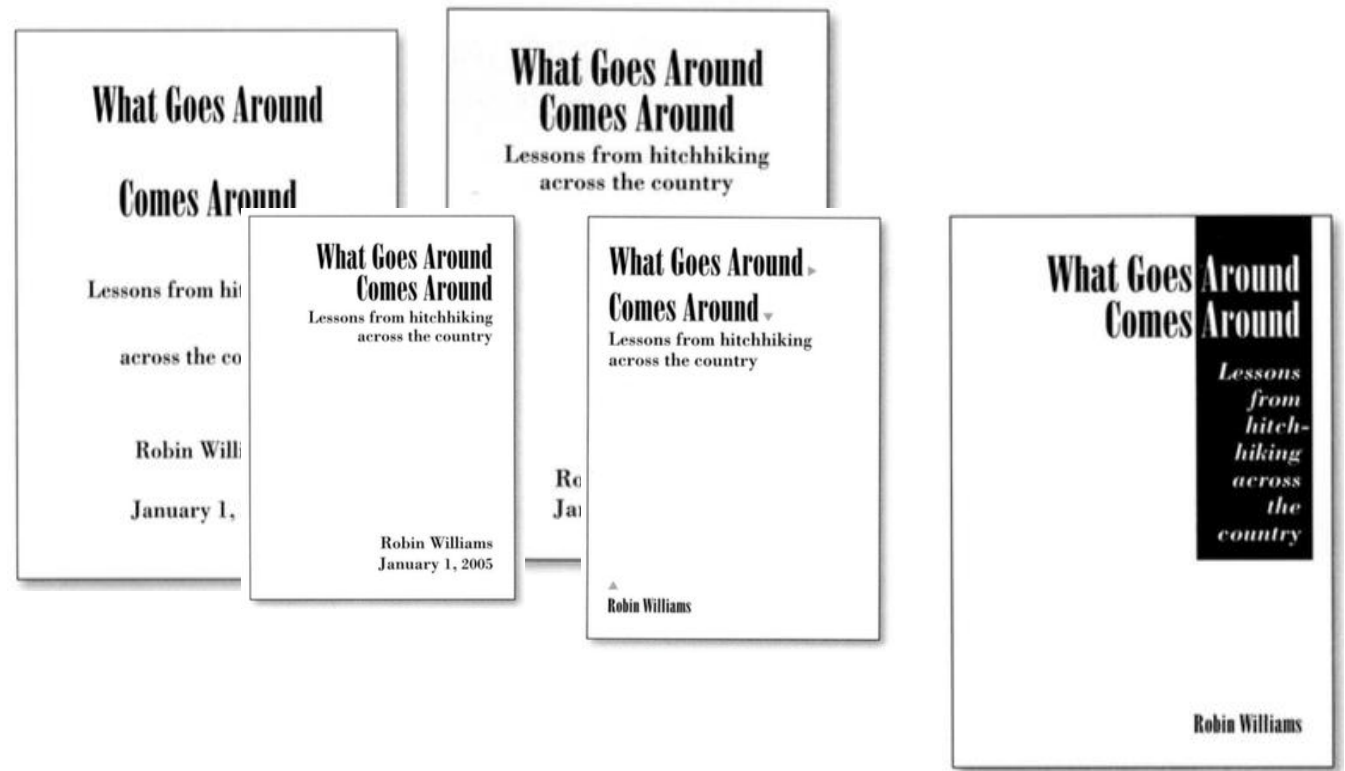
Shape



Form: Basic graphic design ideas

- **proximity**
 - do group related items together
 - avoid equal whitespace between unrelated
- **alignment**
 - do find/make strong line, stick to it
 - avoid automatic centering
- **repetition**
 - do unify by pushing existing consistencies
- **contrast**
 - if not identical, then very different
 - avoid not quite the same

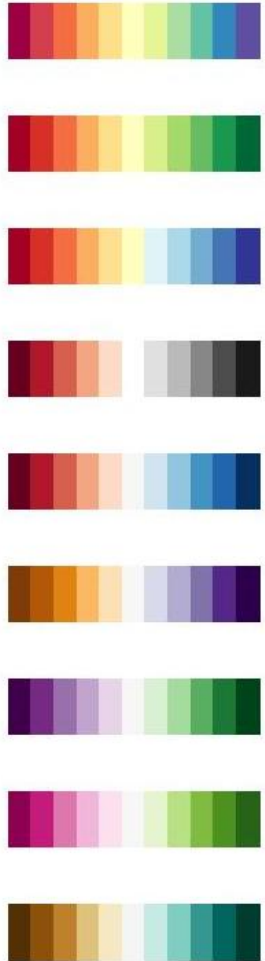
- *The Non-Designer's Design Book, 4th ed. Robin Williams, Peachpit Press, 2015.*
 - fast read, very practical to work through whole thing



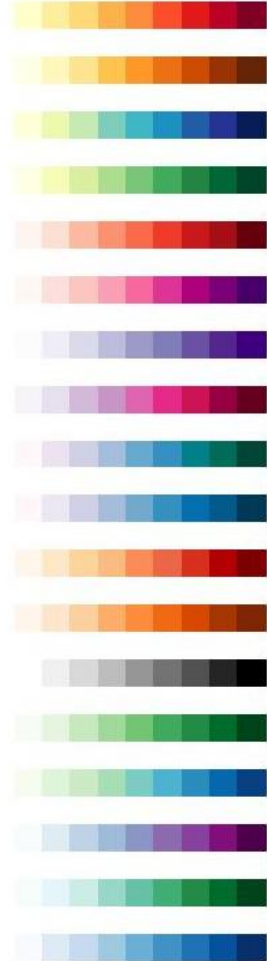
Color palette design considerations: univariate

segmented

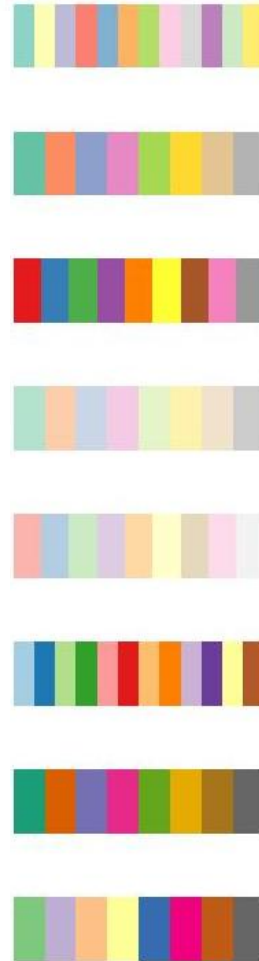
diverging



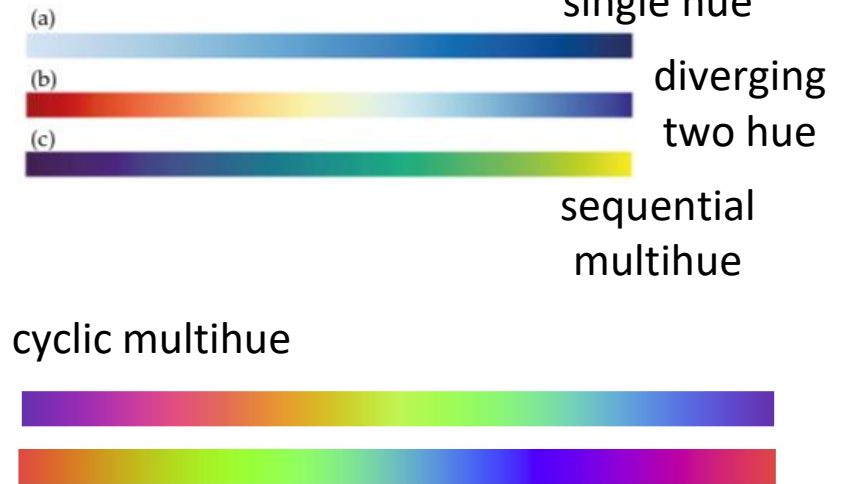
sequential



categorical



continuous



- segmented or continuous?
- diverging or sequential or cyclic?
- single-hue or two-hue or

- multi-hue?
- perceptually linear?
- ordered by luminance?
- colorblind safe?

Colormaps: bivariate

→ Categorical



→ Ordered

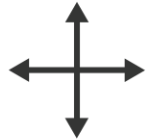
→ *Sequential*



→ *Diverging*

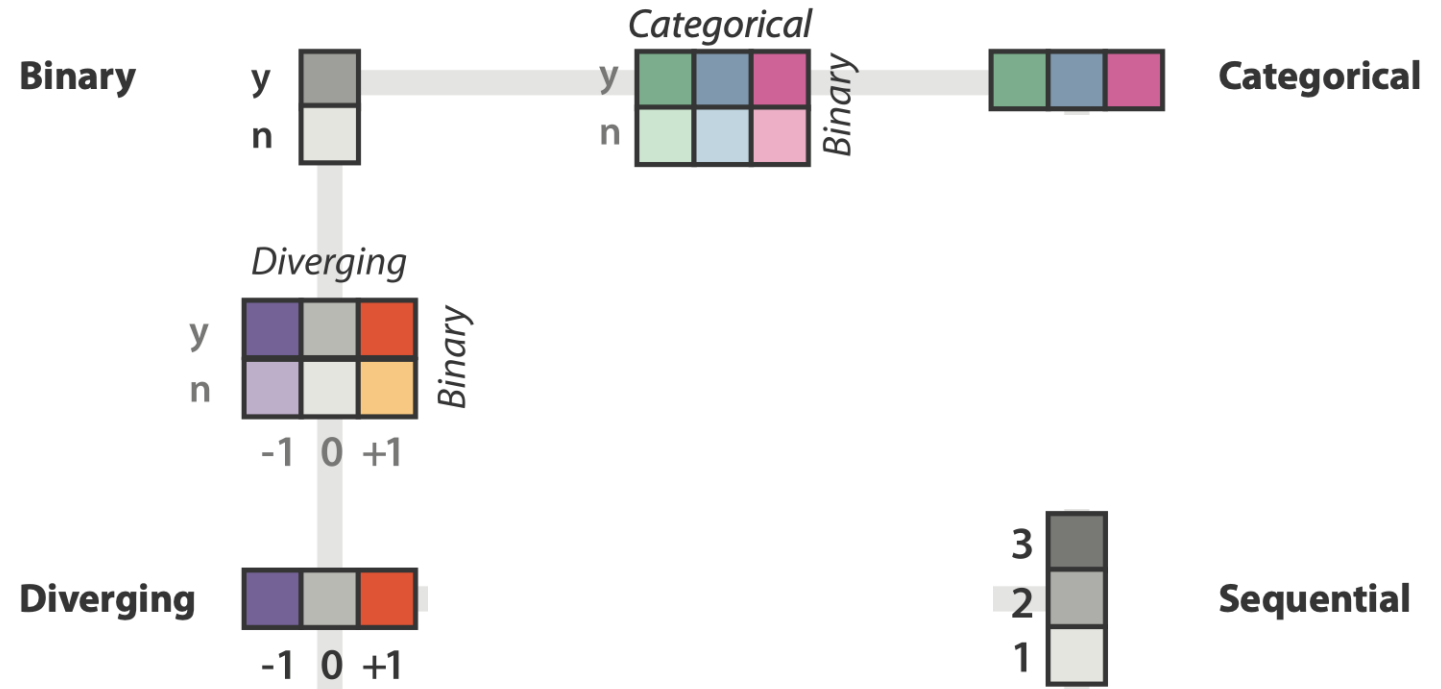


→ Bivariate



- bivariate best case

- binary in one of the directions



binary saturation

categorical hue

Remarks

- Understanding Data Context:
- Choosing Visualization Techniques:
- Interpreting User Intentions:
- Handling Data Quality Issues:
- Aesthetic and Design Sensibilities:
- Interpreting Visual Output:
- Adapting to Diverse Data Types:
- User Interaction and Feedback:
- Automating all the steps of data visualization is a challenging task.
- Incorporating analytics into this process adds another layer of complexity.
- Completely replacing human intervention, however, remains the most difficult aspect, as it requires nuanced understanding, creative decision-making, and contextual analysis.

Related Work

Related Work

Foundation Models for Creative Tasks

- **Advancements in Transformer-Based Models:**
 - Large models trained on diverse datasets (text, images, audio, video).
 - Single models achieving state-of-the-art performance across multiple data modalities.
- **Foundation Models:**
 - Concept introduced by Bommasani et al. (2021).
 - Effective in human creativity tasks, capable of handling varied inputs.
- **Examples of Language Models:**
 - GPT-5 series (Brown et al., 2020), OPT, PALM, LAMBDA.
 - Effective in language tasks like text summarization, question answering.

Related Work

- **Code-Oriented Language Models:**
 - Codex, AlphaCode, InCoder.
 - Excel in code generation and intelligence tasks.
- **Image Generation Models:**
 - CLIP, DALL·E, Latent Diffusion.
 - Capable of tasks like image captioning, editing, and generation.
- **Program-Aided Language Models (Program-Aided LLMs):**
 - Generate programs as reasoning steps.
 - Offload execution to interpreters (e.g., Python).

Related Work

- Each of these steps require expertise, and can be tedious as well as error prone for users with limited visualization experience (novices).
- Existing research has sought to address these challenges by automating the visualization (AUTOVIZ) creation process, given a dataset (Podo et al., 2023).
- Automation may occur in two modes:
 - **Fully automated** - the system automatically generates visualizations relevant to the data
 - **Semi-automated** - the user specifies their goals and the system generates visualizations that address these goals. The former mode is valuable for users unfamiliar with the data and the latter is valuable for users with some familiarity with the data and the visualization task.

Related Work

Approaches in AUTOVIZ:

- Includes **heuristics**, **task decomposition**, and **learning-based approaches**.
- **Heuristics-Based Approaches:**
- Generates a search space of potential visualizations by exploring data properties.
- Ranks visualizations based on quality metrics.
- Examples:
 - **DeepEye** (Luo et al., 2018): Uses a binary classifier to rank visualizations as "good" or "bad".
 - **Voyager** (Wongsuphasawat et al., 2017): Uses heuristics to explore visualization space.
- Limitations: Tedious maintenance, limited coverage, and doesn't leverage information in existing datasets.

Related Work

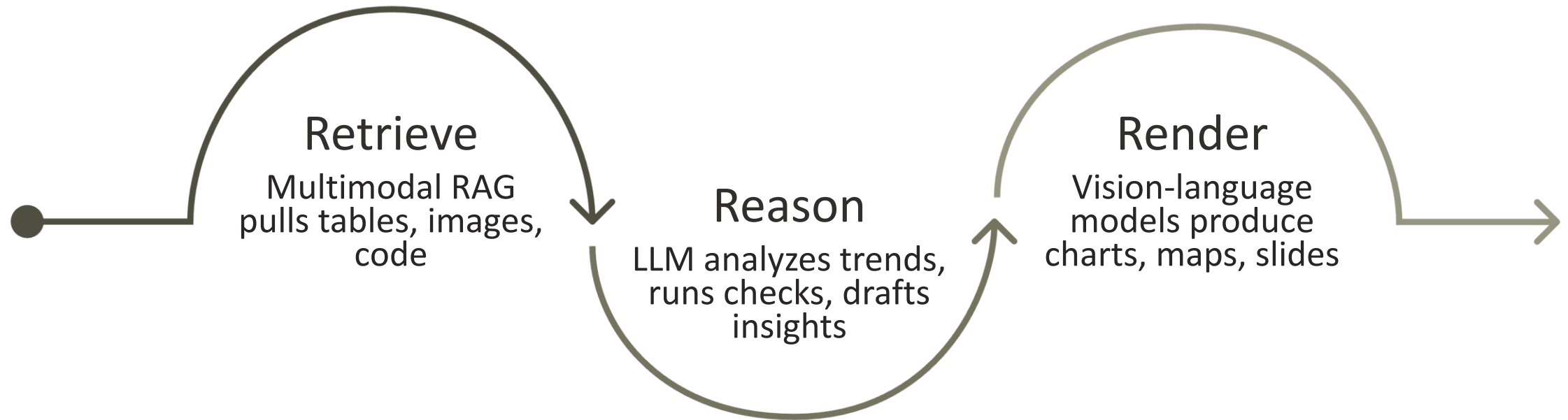
- **Task Decomposition Approaches:**

- Breaks down the AUTOVIZ process into smaller tasks, solved using specialized tools.
- Aggregates the results to create visualizations.
- Example: **NL4DV** (Narechania et al., 2020) parses natural language queries to generate Vega-Lite specifications.
- Limitation: Performance bottlenecks in individual steps, such as challenges in natural language disambiguation.

Related Work

- **Data2Vis** (Dibia and Demiralp, 2019):
 - Uses a sequence-to-sequence model.
 - Translates raw JSON data into Vega-Lite specifications.
 - Learns visualization mappings implicitly.
- **Limitations:**
 - Limited to a single visualization grammar.
 - Requires custom models, paired training data, and specific training objectives.
 - Lacks a method for creating infographics.
 - Does not allow fine-grained control over visual output.
 - Lacks robust error detection and recovery mechanisms.

Core Pipeline of an AI-Assisted Visual Analyst



This end-to-end pipeline — Retrieve, Reason, Render — enables LLMs to go from raw enterprise data to a polished, insight-rich visual in minutes, not hours.

Breakthrough Engines (2025–2026)

1

PPTAgent

Zheng et al., Jan 2025 — edit-based two-stage workflow outperforming prior text-to-slide models on **Content, Design & Coherence**.

2

AutoPresent

Ge et al., CVPR 2025 — 8B LLaMA model generating slide code from natural language, matching closed-source GPT-4-O in visual quality.

3

Prompt-to-Plot

Anand S, June 2025 — end-to-end LLM pipeline that discovers data, writes analysis code, and renders plots with **zero user code**.

4

PPTC Benchmark

Guo et al., 2023 — GPT-4 achieves **75.1%** single-turn accuracy but reveals a **94% drop** in multi-turn session success, guiding future design.

Real-World Impact: Netflix & ViperGPT



Integration

Netflix's internal AI team integrated **ViperGPT** (Oct 2024) to auto-generate quarterly performance dashboards across content acquisition teams.

Efficiency Gained

Analyst effort slashed by **87%** — freeing senior data scientists for higher-order strategic work rather than routine charting.

Business Outcome

\$12M saved in labor costs and a **30% faster** decision cycle for content acquisition — measurable ROI in the first cycle.



Orchestrates multiple vision models for open-world visual reasoning — ideal for dynamic dashboard generation.



Compositional visual reasoning engine for complex diagram synthesis from natural language instructions.



Modular agent that chains LLMs with specialist vision APIs — enabling multi-step visual analysis workflows.



Lightweight vision-language model enabling on-device chart generation — no cloud dependency required.

Call to Action: Pilot AI-Driven Visualization Today



Launch a 30-Day Pilot

Start with a high-impact dataset — such as your sales funnel — and measure insight latency before and after AI integration.

Target Outcomes

Aim for a **30% reduction** in insight latency and an estimated **\$500K ROI** in the first quarter alone.

The Moment Is Now

The tools exist. The benchmarks prove it. **Your next slide could be built by an LLM — let's make it happen.**

Emergence of LIDA

LIDA addresses these limitations in several ways:

- (i) Leverages patterns learned by LLMs from massive language and code dataset, applying this knowledge to subtasks.
- (ii) Provides a single grammar-agnostic pipeline that generates visualization in multiple programming languages and visualization grammars.
- (iii) Supports natural language based control of generated visualizations.
- (iv) leverage emergent capabilities of large language models such chain of thought reasoning to improve reliability of generated text/code, model calibration (predictions on correctness probabilities of visualizations) as well as self-consistency in ranking/filtering results.

The LIDA System

The LIDA System

- LIDA is a novel multi-stage, modular approach for the automatic generation of data visualization and infographics using LLMs:
 - Efficiently represent datasets as **NL summaries**, suitable as grounding context for an LLM to address visualization tasks.
 - Generate a set of **visualization goals** using LLMs.
 - Importantly, it leverage **prompt engineering** to steer the model towards generating correct visualization that follow best practices.
 - LIDA primarily utilizes the OpenAI gpt-3.5-turbo-x line of models for **text and code generation**.

The LIDA System

- Apply LLMs to generate **grammar-agnostic visualization** specification based on generated (or human provided) goals.
- Provide a **hybrid interface** that supports traditional direct manipulation controls (e.g., manually select which fields to explore) and a rich multilingual NL interface to support user's with varied skill/experience.
- Apply **text-conditioned image generation models (IGM)** models in generating stylized infographics that are both informative (generally faithful to data), aesthetically pleasing, memorable and engaging.

The LIDA System

AutoViz Core automated visualization capabilities



Data Summarization

Datasets can be massive. LIDA summarizes data into a compact but information dense natural language representation used as grounding context for all subsequent operations.



Automated Data Exploration

Unfamiliar with a dataset? LIDA provides a fully automated mode that generates meaningful visualization goals based on the dataset. EDA for free.



Grammar-Agnostic Visualizations

Want visualizations created in python in Altair, Matplotlib, Seaborn etc? How about R, C++ ? LIDA is grammar agnostic i.e., can generate visualizations in any grammar represented as code.



Infographics Generation

Convert data into rich, embellished, engaging stylized infographics using image generation models. Think data stories, personalization (brand, style, marketing etc.)?

The LIDA System

VizOps Operations on Generated Visualizations



Visualization Explanation

Get detailed descriptions of visualization code. This has applications in accessibility, data literacy, education, and debugging/sensemaking of visualizations.



Self-Evaluation

LLMs like GPT-3.5 and GPT-4 encode visualization best practices. LIDA applies these capabilities in generating multi-dimensional evaluation scores for visualizations represented as code.



Visualization Repair

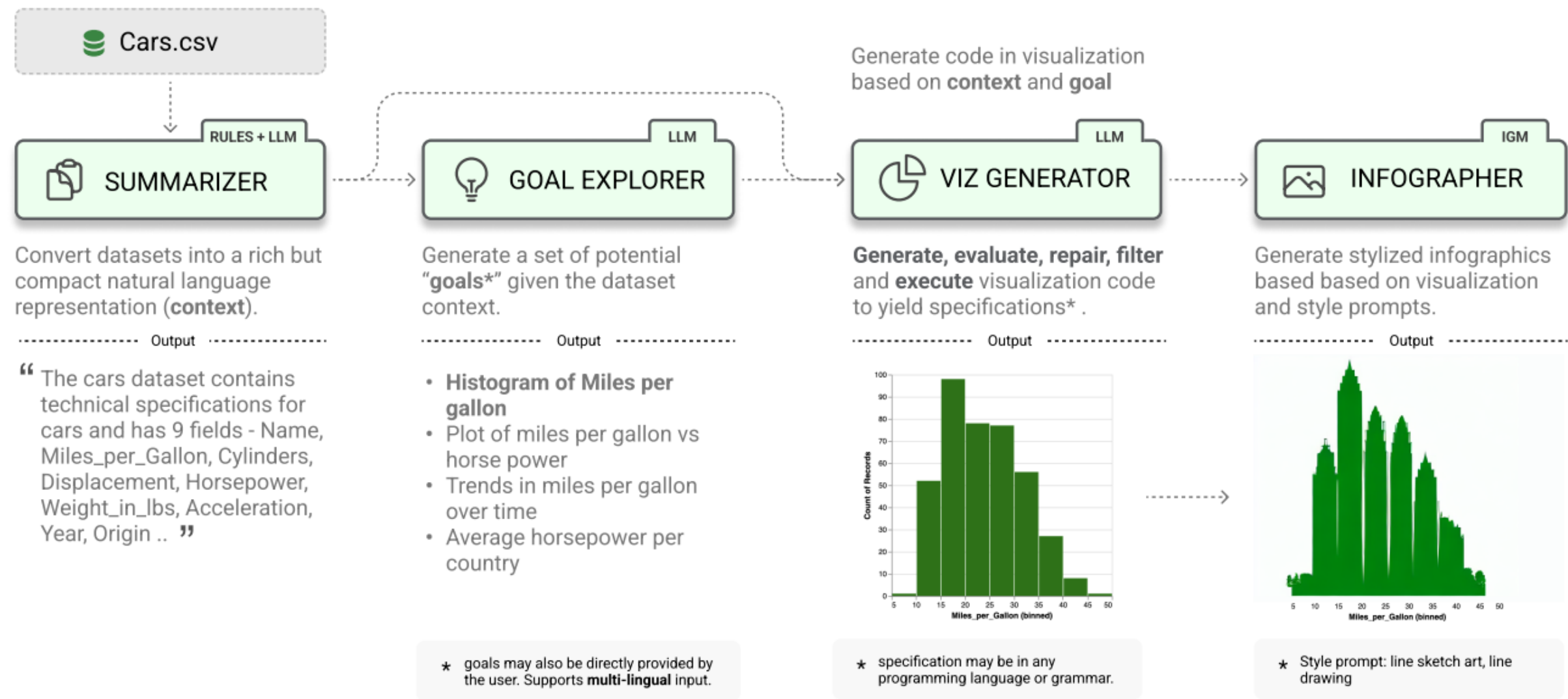
LIDA provides methods to automatically improve visualizations (via self-evaluation feedback) or repair visualizations based on user provided or compile feedback.



Visualization Recommendations

Given some context (goals, or an existing visualization), LIDA can recommend additional visualizations that may be useful to the user (e.g., for comparison, or to provide additional perspectives).

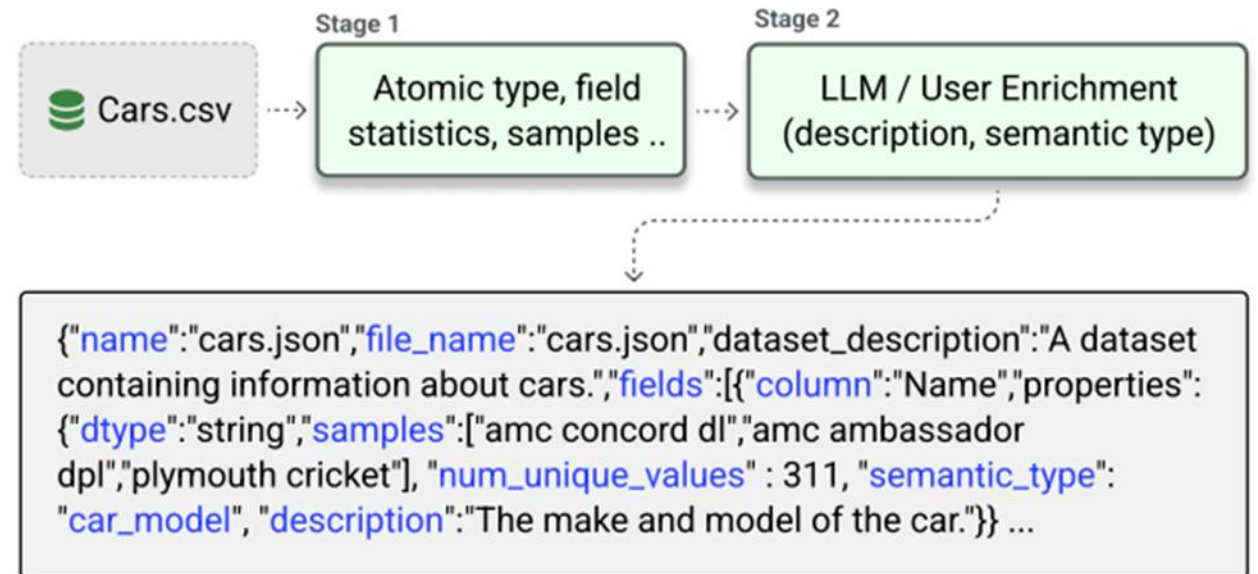
LIDA System Architecture



The LIDA System: SUMMARIZER

- The goal of the summarizer is to produce an information dense but compact summary for a given dataset that is useful as grounding context for visualization tasks.
- A useful context is defined as one that contains information an analyst would need to understand the dataset and the tasks that can be performed on it.

The SUMMARIZER module constructs a NL summary from extracted data properties (atomic types, field statistics) and an optional LLM enrichment (predicted field descriptions, semantic types).



The LIDA System: SUMMARIZER

The summary is implemented in two stages

- **Stage 1 - Base summary generation:**
 - We apply rules in extracting dataset properties including atomic types (e.g., integer, string, boolean) using the pandas library (McKinney, 2010), general statistics (min, max, # unique values) and a random non-null list of n samples for each column.
- **Stage 2 - Summary enrichment:**
 - The base summary is optionally enriched by an LLM or a user via the LIDA UI to include semantic description of the dataset (e.g., a dataset on the technical specification of cars), and fields (e.g., miles per gallon for each car) as well as field semantic type prediction (Zhang et al., 2019).

The LIDA System: GOAL EXPLORER

- This module generates data exploration goals, given a summary generated by the SUMMARIZER.
- Here, goal generation as a multitask generation problem where the LLM must generate a question (hypothesis), a visualization that addresses the question and rationale .

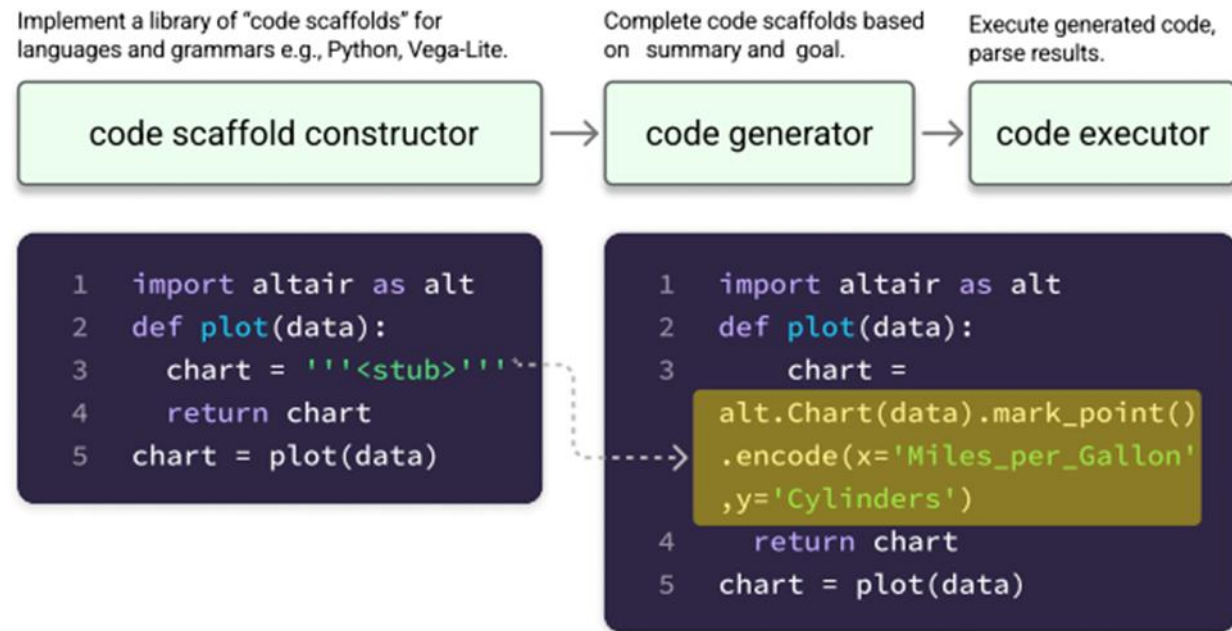
```
{ "question": "What is the distribution of Miles_per_Gallon?",  
  "visualization": "Histogram of Miles_per_Gallon",  
  "rationale": "This tells us about the fuel efficiency of the cars in the  
dataset and how it is distributed." }
```

A goal generated by LIDA is a JSON data structure that contains a question, a visualization and a rationale.

The LIDA System: VISGENERATOR

The VISGENERATOR generates visualization specifications and is comprised of 3 submodules

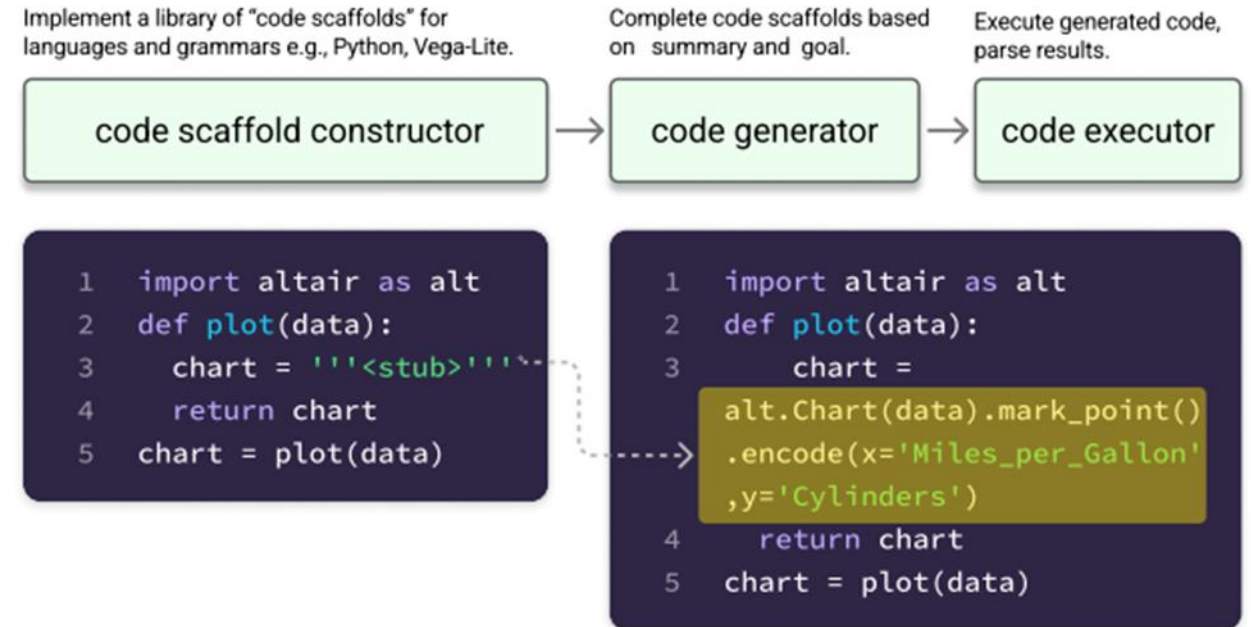
- a code scaffold constructor,
- a code generator and
- a code executor



The LIDA System: VISGENERATOR

Code scaffold constructor:

- Implements a library of code scaffolds that correspond to programming languages and visualization grammars e.g., python scaffolds support grammars such as Matplotlib, GGPlot, Plotly, Altair, Seaborn, and Bokeh.
- Each scaffold is an *executable program* that
 - i.) imports relevant dependencies
 - ii.) defines an empty function stub which returns a visualization specification



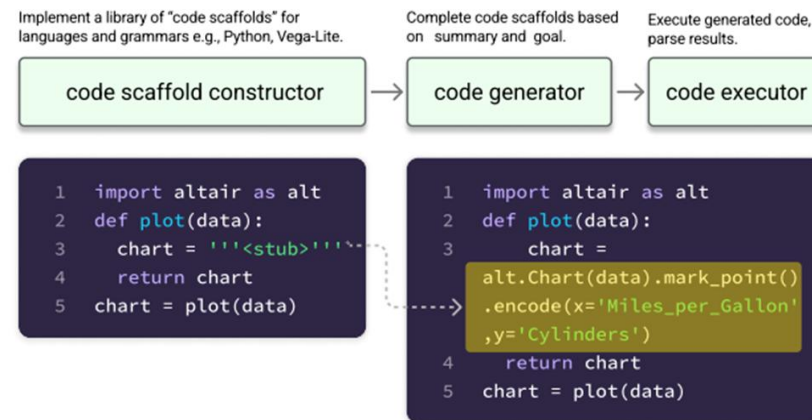
The LIDA System: VISGENERATOR

Code generator:

- Takes a scaffold, a dataset summary, a visualization goal, and builds a prompt.
- An LLM (applied in **fill-in-the-middle mode** (Bavarian et al., 2022)) is then used to generate the candidate visualization code specifications.

How Fill-in-the-Middle Works:

- **Input Structure:** The input is typically divided into three parts:
 - **Prefix:** Text that comes before the gap.
 - **Suffix:** Text that comes after the gap.
 - The **Gap** is the part where the model needs to generate content.
- **Model Objective:** The LLM generates text that connects the prefix and suffix naturally, ensuring consistency with the surrounding content.



The LIDA System: VISGENERATOR

Code executor: Post-processes and execute the code specifications as well as filters the results.

- LIDA implements several filtering mechanisms to detect errors, each with latency tradeoffs:
 - Generates a large sample for n with high temperature, discard candidates that do not compile.
 - Apply self consistency in LLMs where multiple candidates are generated and the solution with the highest consensus is selected.
 - Generate correctness probabilities for all candidates and selects the one with the highest probability.

```
1  # pip install lida
2
3  from lida.modules import Manager
4
5  lida = Manager()
6  summary = lida.summarize("data/cars.csv")
7  goals = lida.generate_goals(summary, n=1)
8
9  vis_specs = manager.generate_viz( summary=summary,
10                                   goal=goals[i])
11  charts = manager.execute_viz( code_specs=vis_specs,
12                                data=manager.data, summary=summary)
13  print(charts)
```


Visualization Generation

Select a goal above or describe a new visualization goal to generate a visualization.

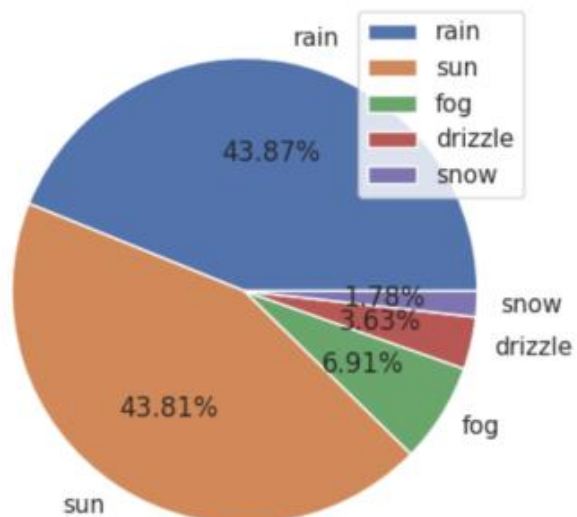
What is the distribution of weather conditions observed as a pie chart

Generate

What is the distribution of weather conditions observed as a pie chart

What is the distribution of weather conditions observed as a pie chart

What is the distribution of weather conditions observed as a pie chart



How was this visualization created? See the specifications and code below.

Python Code

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

def plot(data: pd.DataFrame):
    weather_counts = data['weather'].value_counts()
    plt.pie(weather_counts, labels=weather_counts.index, autopct=t='%.2f%')
    plt.title("What is the distribution of weather conditions observed as a pie chart", wrap=True)
    plt.legend(weather_counts.index, loc="best")
    return plt;
```

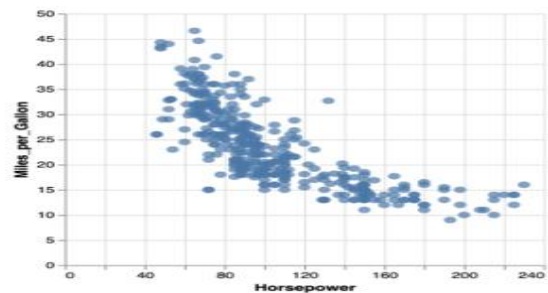
Refine

Explain

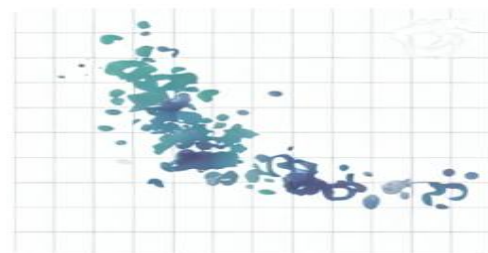
Evaluate

Recommend!

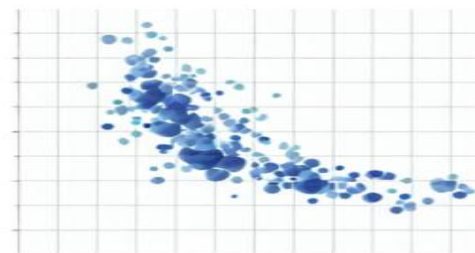
Reference Visualization



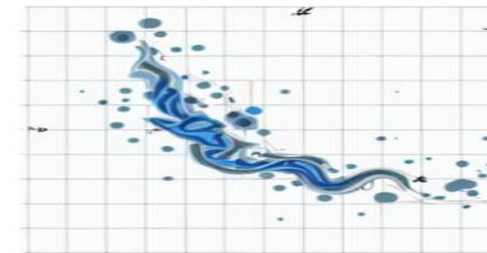
Generated stylized infographics



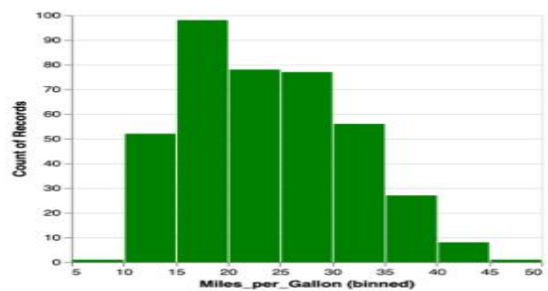
underwater art, green



pastel art



underwater art



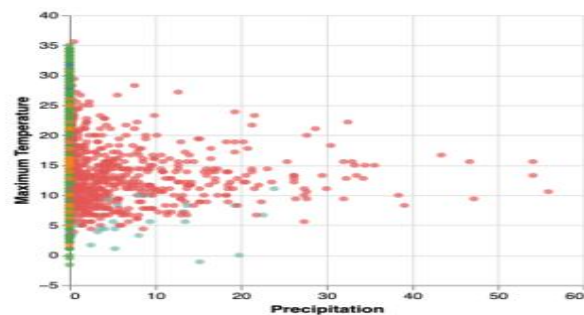
3d art, white background



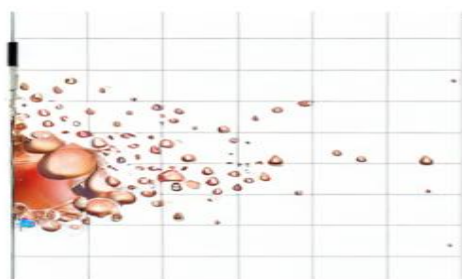
3d art, skyscraper, hongkong



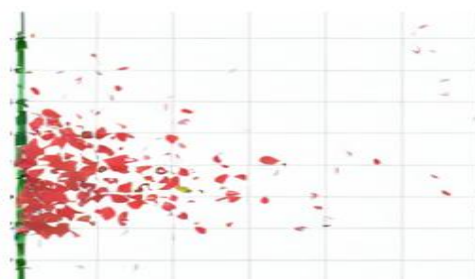
sketch art, cross hatching



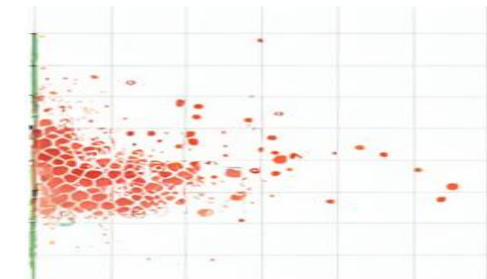
Weather Type
 ● drizzle
 ● fog
 ● rain
 ● snow
 ● sun



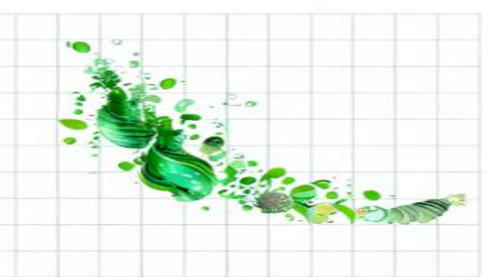
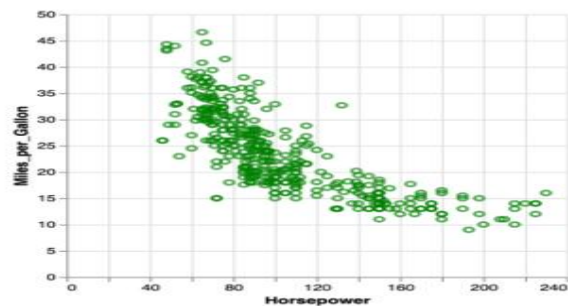
rain drops



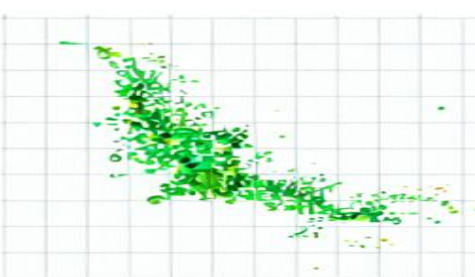
leaves, green leaves



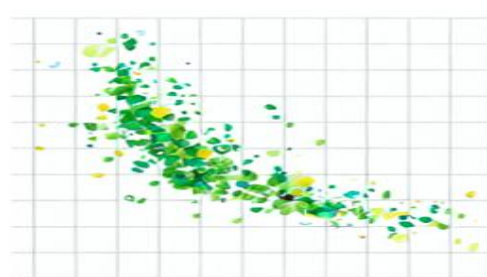
underwater art, seaweed



underwater art, shells



pastel art



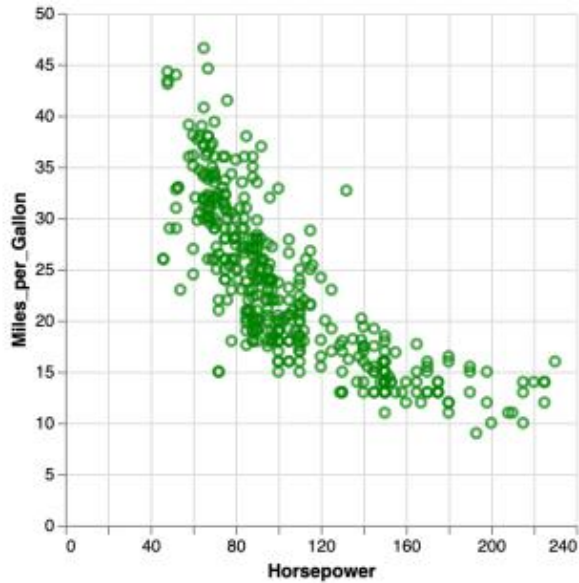
oil on canvas, impasto

INFOGRAPHER

- This module is tasked with generating stylized graphics based on output from the VISGENERATOR module.
- It implements a library of visual styles described in NL that are applied directly to visualization images.
- This style library is editable by the user.
- These styles are applied in generating infographics using the **text-conditioned image-to-image generation** capabilities of diffusion models (Rombach et al., 2022), implemented using the **Peacasso library API** (Dibia, 2022).

Infographics

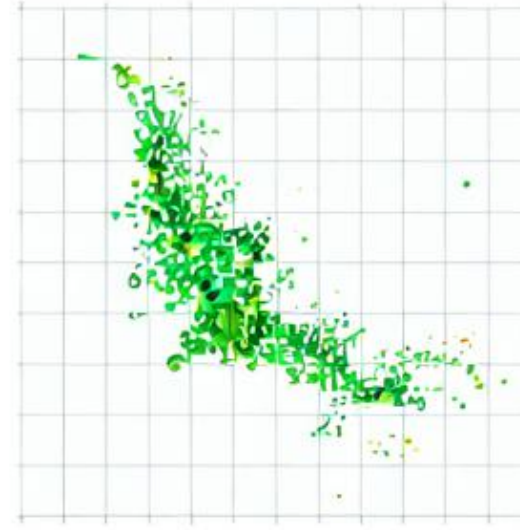
Reference Visualization



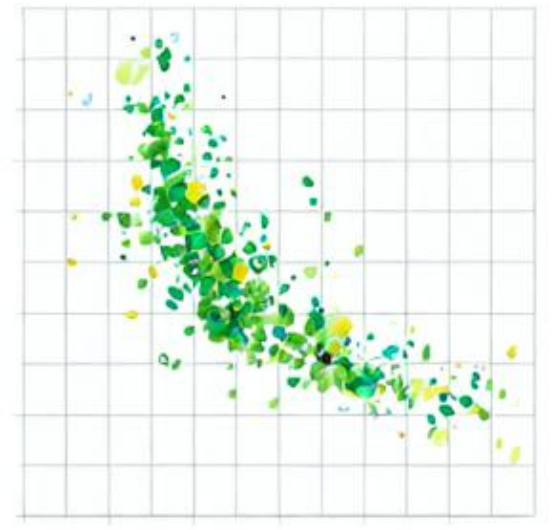
Generated stylized infographics



underwater art, shells



pastel art



oil on canvas, impasto

USER INTERFACE

LIDA implements a user interface that communicates with the core modules over a REST and Websocket API. The user interface implements several views.

Data upload and summarization:

- This view allows the user to upload a dataset and explore a sample of rows in the dataset via a table view.
- A data upload event triggers a call to the SUMMARIZER and GOAL EXPLORER module and displays a summary of the dataset and a list of potential goals.
- This view also allows the user to optionally annotate and refine the generated summary or curate fields used in the dataset.

USER INTERFACE

- **Data upload and summarization:** This view allows the user to upload a dataset and explore a sample of rows in the dataset via a table view.
- **Visualization view:** This view allows the user to optionally provide a visualization goal in NL (e.g., "what is the fuel efficiency per country?") or select a generated goal and then displays a generated visualization .
- For each visualization, intermediate output from the models (underlying data summary, visualization specification, code scaffold) are shown as explanations to aid in sensemaking, and debugging.

Select a visualization library/grammar

Seaborn



Altair

Matplotlib

Seaborn

GGPlot

⚙️ Generation Settings

Model: **gpt-4-0314**, n: 1, number of Temperature: 0 ...



Click or drag file to this area to upload
Upload .json or .csv files to generate a visualization.

📁 Don't have data? Try any of the files below

stocks.csv

cars.json

wheat.json

movies.json

seattle-weather.csv

sp500.csv

📄 Data Summary

An enriched representation of the data (with predicted semantic types and descriptions)

seattle-weather.csv This dataset contains weather data for Seattle from 2012 to 2015.

^ data summary | seattle-weather.csv



Date

date | date

The date of the weather observation.

Unique values: 1461

min: 2012-01-01 , max: 2015-12-31

View samples

Precipitation

number | precipitation

The amount of precipitation in inches.

Unique values: 111

0 , max: 55.9 | std: 6.680

View samples

Temp_max

number | temperature

The maximum temperature in degrees Fahrenheit.

Unique values: 67

min: -1.6 , max: 35.6 | std: 7.350

View samples

Temp_min

number | temperature

The minimum temperature in degrees Fahrenheit.

Unique values: 55

min: -7.1 , max: 18.3 | std: 5.023

View samples

Wind

number | wind_speed

The wind speed in miles per hour.

Unique values: 79

min: 0.4 , max: 9.5 | std: 1.438

View samples

Weather

category | weather_condition

The weather condition observed.

Unique values: 5

View samples

View raw summary?

The Evaluation System

Evaluation Metrics

Visualization Error Rate (VER)

- Visualization error rate is computed as the percentage of generated visualizations that result in code compilation errors.
- This metric provides critical insights into the reliability of the LIDA pipeline and impact of changes to the system (e.g., prompt engineering or scaffold update).
- Where: - E = Number of generated visualizations with code compilation errors, and - T = Total number of generated visualizations.

$$\text{VER} = \frac{E}{T} * 100$$

Evaluation Metrics

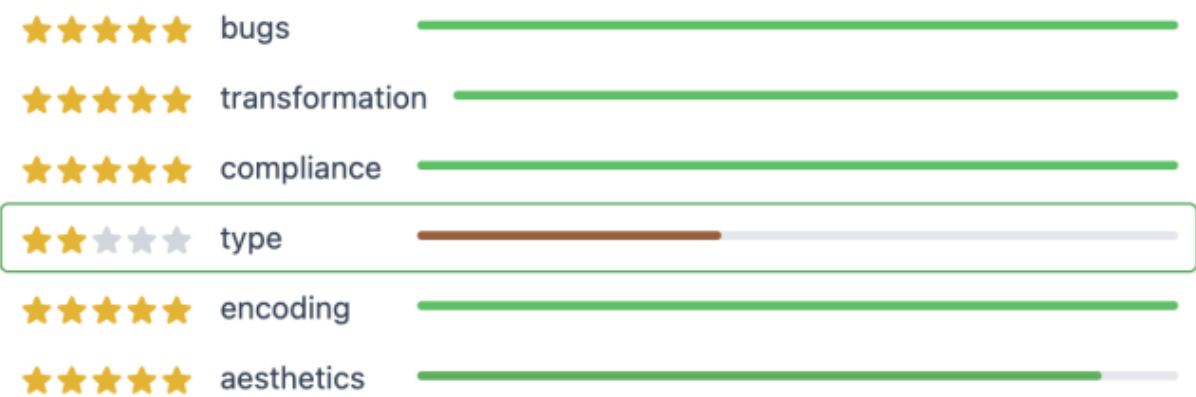
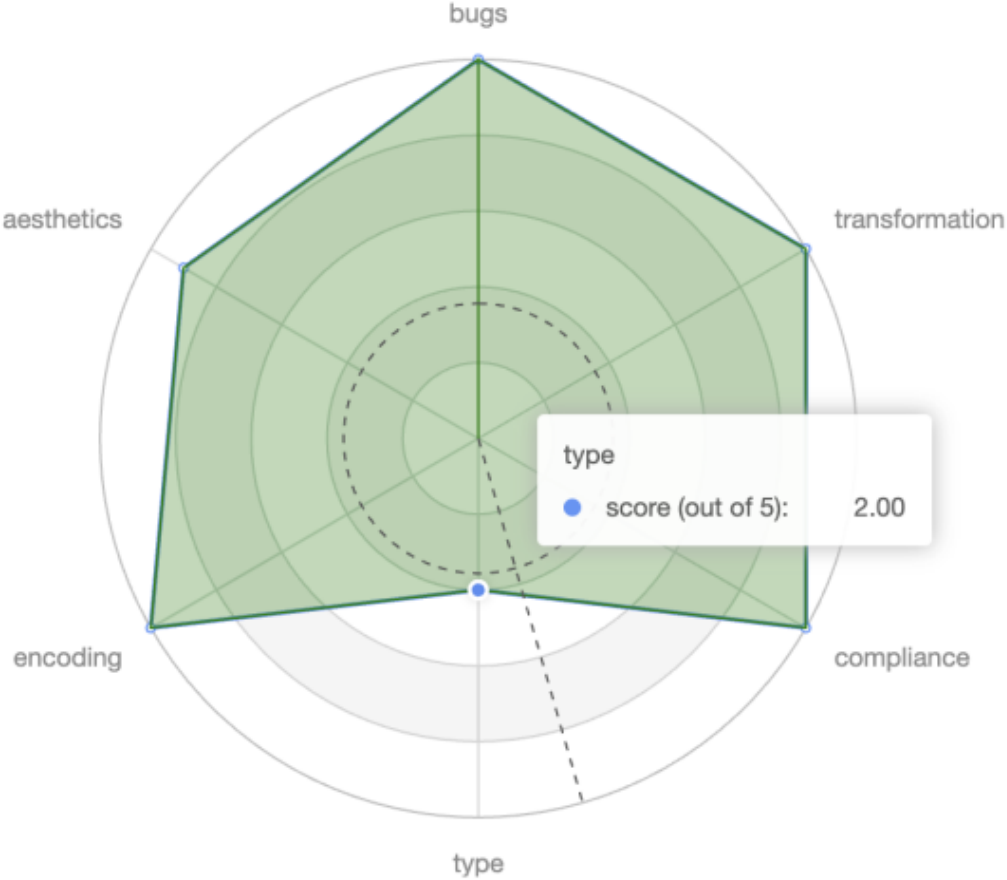
Self-Evaluated Visualization Quality (SEVQ)

- SEVQ metric evaluates visualizations using GPT-4.
 - Scores visualizations on a scale of 1-10 with rationale.
-
- Six Dimensions of Visualization Quality
 - **Code Accuracy:** Does the code correctly generate the desired visualization?
 - **Data Transformation:** How well is the data transformed for visualization?
 - **Goal Compliance:** Does the visualization meet the intended goal?
 - **Visualization Type:** Is the appropriate visualization type chosen?
 - **Data Encoding:** How effectively is the data represented visually?
 - **Aesthetics:** Is the visualization aesthetically pleasing?

Use the evaluate visualization button below to generate self-evaluation scores for the chart.

Evaluate the chart.

The visualization has been evaluated across multiple dimensions. Select a dimension to view rationale for the score



Type

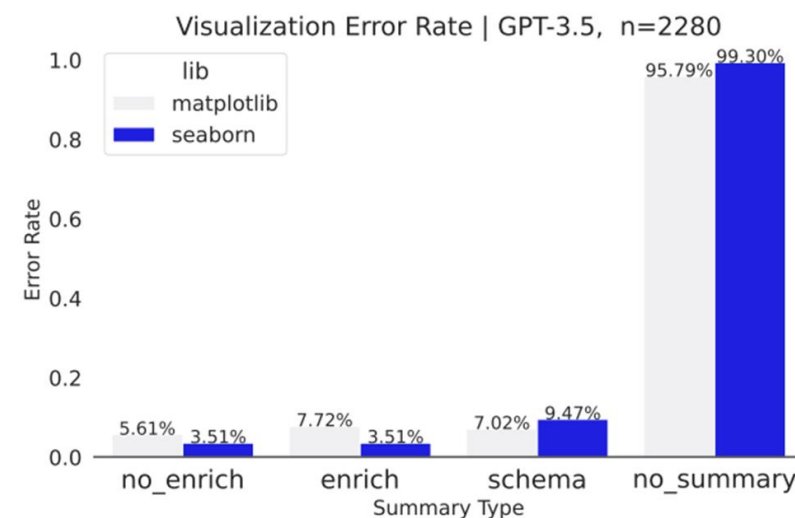
2.0 out of 5

While a pie chart can show the distribution of weather conditions, it is not the best choice for this type of data. A bar chart would be more effective in conveying insights, as it allows for easier comparison of categories and is less prone to misinterpretation.

Auto Repair Chart.

Evaluation Benchmark Settings

- **Data Source:** 57 datasets from the Vega repository.
- **Task:** LIDA generates 5 goals and 1 visualization per goal across multiple grammars.
- **Settings** for reproducibility:
 - **Temperature:** 0 (deterministic results)
 - **Samples (n):** 1 per goal
- **Visualization Gallery:** Available on the LIDA project page.
 - (i) no_enrich: a base summary with no enrichment
 - (ii) enrich: summary with LLM enrichment,
 - (iii) schema: only field names, i.e., schema as summary, and
 - (iv) no_summary: no summary.



Installation Requirements

1. Set Up Conda Environment:

Create a new environment:

```
conda create -p venv python==3.10 -y
```

Activate the environment:

```
conda activate venv/
```

2. Install Packages:

Install LIDA:

```
pip install -U lida
```

Install llmx and openai:

```
pip install -U llmx openai
```

3. Configure OpenAI API Key:

- Linux/macOS: `export OPENAI_API_KEY=<your key>`
- Windows: `set OPENAI_API_KEY=<your key>`

4. Start LIDA UI:

- Launch UI on port 8080:
 - `lida ui --port=8080 --docs`

5. Access LIDA UI:

- Open in browser:
 - `http://localhost:8080/`

Installation Requirements

Post-Setup (Future Use):

1. Activate Environment:

- `conda activate venv/`

2. Set API Key (if not set):

- Linux/macOS: `export OPENAI_API_KEY=<your key>`
- Windows: `set OPENAI_API_KEY=<your key>`

3. Restart LIDA UI:

- `lida ui --port=8080 --docs`

4. Open in Browser:

- `http://localhost:8080/`

DEMONSTRATION

References

- Github : <https://github.com/microsoft/lida>
- Paper: <https://arxiv.org/abs/2303.02927>
- Tutorial Notebook:
<https://github.com/microsoft/lida/blob/main/notebooks/tutorial.ipynb>
- Project Page: <https://microsoft.github.io/lida/>

References

- Bavarian, M., Jun, H., Tezak, N., Schulman, J., McLeavey, C., Tworek, J., & Chen, M. (2022). Efficient training of language models to fill in the middle. arXiv preprint arXiv:2207.14255.
- Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosse-lut, A., Brunskill, E., et al. (2021). On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. (2022). Palm: Scaling language modeling with pathways. arXiv preprint arXiv:2204.02311.
- Cohen, A. D., Roberts, A., Molina, A., Butryna, A., Jin, A., Kulshreshtha, A., Hutchinson, B., Zevenbergen, B., Aguera-Arcas, B. H., Chang, C. C., et al. (2022). Lamda: Language models for dialog applications. arXiv.
- Dibia, V. (2022). Interaction design for systems that integrate image generation models: A case study with Peacasso.
- Dibia, V., & Demiralp, Ç. (2019). Data2vis: Automatic generation of data visualizations using sequence-to-sequence recurrent neural networks. IEEE Computer Graphics and Applications, 39(5), 33–46.
- Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., Zhong, R., Yih, W.-t., Zettlemoyer, L., & Lewis, M. (2022). InCoder: A generative model for code infilling and synthesis. arXiv preprint arXiv:2204.05999.
- Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., & Neubig, G. (2022). PAL: Program-aided language models. arXiv preprint arXiv:2211.10435.
- Haroz, S., Kosara, R., & Franconeri, S. L. (2015). Isotype visualization: Working memory, performance, and engagement with pictographs. Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems, 1191–1200.
- Harrison, L., Reinecke, K., & Chang, R. (2015). Infographic aesthetics: Designing for the first impression. Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems, 1187–1190.
- Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield Dodds, Z., DasSarma, N., Tran-Johnson, E., et al. (2022). Language models (mostly) know what they know. arXiv preprint arXiv:2207.05221.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large language models are zero-shot reasoners. arXiv preprint arXiv:2205.11916.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. (2022). Competition-level code generation with AlphaCode. arXiv preprint arXiv:2203.07814.

References

- Lin, S., Hilton, J., & Evans, O. (2022). Teaching models to express their uncertainty in words. *arXiv preprint arXiv:2205.14334*.
- Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., & Zhu, C. (2023). Gpteval: NLG evaluation using gpt-4 with better human alignment. *arXiv preprint arXiv:2303.16634*.
- Luo, Y., Qin, X., Tang, N., Li, G., & Wang, X. (2018). DeepEye: Creating good data visualizations by keyword search. *Proceedings of the 2018 International Conference on Management of Data, SIGMOD*, 1733–1736.
- Narechania, A., Srinivasan, A., & Stasko, J. (2020). NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics*, 27(2), 369–379.
- OpenAI. (2023). GPT-4 technical report.
- Podo, L., Prenkaj, B., & Velardi, P. (2023). Machine learning for visualization recommendation systems: Open challenges and future directions. *arXiv preprint arXiv:2302.00569*.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. (2021). Learning transferable visual models from natural language supervision. *International Conference on Machine Learning*, 8748–8763.
- Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022). Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*.
- Ramesh, A., Pavlov, M., Goh, G., Gray, S., Voss, C., Radford, A., Chen, M., & Sutskever, I. (2021). Zero-shot text-to-image generation. *International Conference on Machine Learning*, 8821–8831.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. *CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10674–10685.
- Satyanarayan, A., Moritz, D., Wongsuphasawat, K., & Heer, J. (2017). Vega-lite: A grammar of interactive graphics. *IEEE TVCG (Proc. InfoVis)*.
- Shi, Y., Liu, P., Chen, S., Sun, M., & Cao, N. (2022b). Supporting expressive and faithful pictorial visualization design with visual style transfer. *IEEE Transactions on Visualization and Computer Graphics*, 29(1), 236–246.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). Llama: Open and efficient foundation language models.
- Tyagi, A., Zhao, J., Patel, P., Khurana, S., & Mueller, K. (2021). User-centric semi-automated infographics authoring and recommendation. *arXiv preprint arXiv:2108.11914*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., & Zhou, D. (2022a). Self-consistency improves chain of thought reasoning in language models. *arXiv preprint